Problem Set 2

CMSC 27200: Theory of Algorithms

Assigned January 16, Due January 24

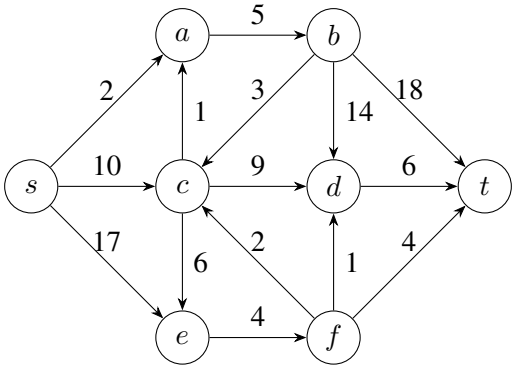
Problem 0: Collaborators and Outside Sources

Who did you collaborate with?

Did you use any outside sources? If so, which sources did you use?

Problem 1: Dijkstra's Algorithm

Use Dijkstra's algorithm to find the shortest path from s to t in the following graph G.



For your answer, first give the state of the priority queue after we add in the edges going out from s. Then give the following data for each new vertex that is reached.

1. The name of the vertex v which is reached and the length of the shortest path from s to v.
2. The state of the priority queue after we add in the edges which go out from v.

After giving this data, give all of the shortest paths from s to t.

Note: For this problem, you do not need to show the heap structure of the priority queue. Instead, you can represent the priority queue as a sorted list. Also, you may use either a priority queue with edges or a priority queue with vertices (where the value for a vertex decreases when a shorter path to that vertex is found).
Note: To find all of the shortest paths from s to t, it is useful to record all of the edges $(u, v) \in E(G)$ which can be the final edge of a minimum length path from s to a given vertex v.

Problem 2: Asset Liquidation

Let's say that you need to sell n assets over n months in order to raise money for a big investment. However, selling the assets takes time so you can only sell one asset per month. Each asset has a selling price p_i and a monthly revenue r_i so if you sell asset i during month j, you will earn a total of $p_i + (j-1)r_i$ from asset i. In which order should you sell the assets in order to maximize your earnings?

Note: We can state this problem mathematically as follows: Find an ordering $S = (i_1, \ldots, i_n)$ of $[n]$ which maximizes $\sum_{j=1}^{n} (p_{i_j} + (j-1)r_{i_j})$.
Note: We assume that for all $i \in [n]$, $p_i \geq nr_i$ so that we always want to sell an asset each month.

(a) Give the order in which you should sell your assets.

(b) Given another ordering $S = (i_1, \ldots, i_n)$ which differs from your ordering, give a swap which can be made which makes S closer to your ordering and prove that this swap either increases or does not change the total earnings from your assets.

(c) Using this, give a rigorous proof that your ordering is optimal. Two ways of doing this are as follows.

1. Consider an ordering such as the lexicographic ordering on possible solutions and prove that the first optimal solution in this ordering must be your solution.
2. Prove by induction that for all $j \in [n]$, there is an optimal ordering which matches the first j assets in your ordering.

(d) What is the runtime for determining your ordering?

Note: For the grading, parts b and c will be considered together as the proof/explanation for your algorithm. As described in the grading rubric for greedy algorithms, you can earn an exemplary grade here.

Problem 3: Supervisory Committee (Rephrasing of Problem 15 in Chapter 4 of Kleinberg-Tardos

You have n part-time employees, each of whom work during an interval $[a_i, b_i]$. You don't have time to supervise all of your employees, so you would like to promote a few of your employees to managers to help supervise the remaining employees. Your employees don't need to be supervised all of the time, but you would like every employee to overlap with at least one of your managers. What is the minimum number of managers you need to accomplish this and how can you choose these managers?
Note: We can express this problem mathematically as follows: What is the minimum size of a subset $I \subseteq [n]$ such that for all $j \in [n]$ there exists an $i \in I$ such that $[a_i, b_i] \cap [a_j, b_j] \neq \emptyset$?

(a) Give a greedy algorithm for this problem.

(b) Give a measure by which this greedy algorithm stays ahead.

(c) Give a rigorous proof that the greedy algorithm stays ahead according to this measure. This proof should either be a proof by induction or a proof by contradiction where we consider the first time that the greedy algorithm falls behind and derive a contradiction.

(d) Explain how to efficiently implement your algorithm (if you did not already do so in part a) and analyze its runtime.

A hint is available for this problem.
Note: For the grading, parts b and c will be considered together as the proof/explanation for your algorithm. As described in the grading rubric for greedy algorithms, you can earn an exemplary grade here.

Problem 4: Minimum Spanning Trees After Modifying an Edge

Let $G = (V, E)$ be an undirected, connected graph where each edge $e \in E$ has an edge length l_e and these edge lengths $\{l_e : e \in E\}$ are all distinct. Let $n = |V|$ be the number of vertices of G, let $m = |E|$ be the number of edges of G, and let T be the minimum spanning tree of G. In this problem, we consider how T may be affected if the length of an edge $e \in E$ changes.

(a) Let's say that the length of some edge $e \in E$ decreases. Give an $O(n)$ time algorithm to find the new minimum spanning tree T' (assume that you know the original minimum spanning tree T).

(b) Give a proof/explanation for why your algorithm is correct.

(c) Let's say that the length of some edge $e \in E$ increases. Give an $O(m)$ time algorithm to find the new minimum spanning tree T' (assume that you know the original minimum spanning tree T).

(d) Give a proof/explanation for why your algorithm is correct.

A hint is available for this problem.
Note: You can earn an exemplary grade for parts b and d by giving a rigorous proof that your algorithm is correct. For this problem, parts b and d have half the usual weight (i.e., an Exemplary grade earns $\frac{1}{2}$ of a point, a Needs Improvement Grade only loses $\frac{1}{2}$ of a point, and an Unsatisfactory grade only loses 1 point).

Hints

3. Working from the beginning of the work day, which employee needs to be supervised first? Which supervisor should you choose for this employee?

4. Recall the two characterizations for when an edge e is in T (since these characterizations were covered in class and are proved in the lecture notes, you do not need to prove them):

1. $e \in T$ if and only if there is a cut (L, R) such that e is the shortest edges between L and R.
2. $e \in T$ if and only if there is no cycle C such that e is the longest edge of C.